OBJECT ORIENTED PROGRAMMING

Unit-IV

Generics – Collections - Utility Packages – Input Output Packages - Inner Classes - Java Database Connectivity – Java Security

PART-A - 2 Marks

1. **What is an Applet?**
   An applet is a special kind of Java program that runs in a Java enabled browser. This is the first Java program that can run over the network using the browser. Applet is typically embedded inside a web page and runs in the browser and works at client side. An applet is embedded in an HTML page using the APPLET or OBJECT tag and hosted on a web server.
2. **What do you understand about RMI?**
   The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. It is a mechanism that allows an object residing in one system (JVM) to access/invoke an object running on another JVM.The RMI provides remote communication between the applications using objects. It is provided in the package **java.rmi**.
3. **What is am inner class?**
   Java inner class or nested class is a class which is declared inside the class or interface. Inner class means one class which is a member of another class. We use inner classes to logically group classes and interfaces in one place so that it can be more readable and maintainable.
4. **State the use of JDBC driver.**
   JDBC driver is used to communicate with a data source through JDBC. Whenever need a JDBC driver that intelligently communicates with the respective data source. JDBC Driver is a software component that enables java application to interact with the database. It acts as a middle layer interface between java applications and database.
5. **Which Java, Util classes and interfaces support event handling?**
   EventObject class and the EventListener interface support event handling in Java.Util class.
6. **List the difference between Java inner class and anonymous class.**

Inner class	Anonymous class
Inner class has a class name.	An anonymous class has no name declared for the class.
Inner class consists of a class declared within a method.	An anonymous class is declared when an instance is created.

7. **What is local inner class and anonymous inner class? Give their advantages.**
   Anonymous inner class is a type of inner class which has no name and it can be instantiated only once. It does not have a constructor because it does not have a class name and cannot be static Anonymous inner class always extend a class or implement an interface.
   A class is created inside a method of another class is called **local inner class**. To invoke the methods of local inner class, you must instantiate this class inside the method.

Advantages of inner class

Nested classes are used to develop more readable and maintainable code because it logically group classes and interfaces in one place only. Code Optimization: It requires less code to write.

Advantages of Anonymous inner class: In anonymous classes, objects are created whenever they are required. That is, objects are created to perform some specific tasks. n object of the anonymous class is created dynamically when we need to override the display() method. Anonymous classes also help us to make our code concise.

8. Name some Java API Packages.

- **java.applet**
- **java.awt**
- **java.awt.color**
- **java.sql**
- **java.util**

9. What is JDBC?

JDBC is an API(Application programming interface) which is used in java programming to interact with databases.
The classes and interfaces *of JDBC allows application to send request made by users to the specified database* JDBC is a Java API to connect and execute the query with the database. It is a part of JavaSE (Java Standard Edition).

10. What are Utility Packages?

utils.compression	Useful for geometry compression.
utils.geometry	Includes geometry utilities, such as stripification, normal generation, tessellation, and primitive construction.
utils.image	Useful for loading Java 3D texture objects.
utils.picking	Useful for defining picking operations and retrieving information about the picked object.
utils.picking.behaviors	Combines picking with mouse-based rotation, translation, and zoom behaviors.
utils.universe	Useful for setting up a user environment to get a Java 3D program up and running quickly and easily.

11. Define a generic class with an example.

Generics means **parameterized types. Java Generics** is a set of related methods or a set of similar types. Generics allow types Integer, String, or even user-defined types to be passed as a parameter to classes, methods, or interfaces. Generics are mostly used by classes like HashSet or HashMap.

Example:

```
class MyGen<T>{
T obj;
void add(T obj){this.obj=obj;}
```

```
T get(){return obj;}
}
```

12. What is an inner class? Give an example.

Inner class are defined inside the body of another class (known as **outer class**). These classes can have access modifier or even can be marked as abstract and final. Inner classes have special relationship with outer class instances. This relationship allows them to have access to outer class members including private members too.

Example:

```
OuterClass outerObject = new OuterClass(); OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

13. What is dynamic binding?

The binding which occurs during runtime is called **dynamic binding in java**. This binding is resolved based on the type of object at runtime.

14. What are wrapper classes?

The **wrapper class in Java** provides the mechanism *to convert primitive into object and object into primitive*.

15. What is the difference between system.out.print and system.out.println in java?

System.out.println(" "); prints the string(s) to the next line and goes to the next line again at the end of it.System.out.print(" "); prints the string(s) right after each other without going to the next line.

16. What is Object Serialization?

Serialization in Java is a mechanism of *writing the state of an object into a byte-stream*. The serialization process is platform-independent. Serialization in Java allows us to convert an Object to stream that we can send over the network or save it as file or store in DB for later usage.

17. Difference inner class and nested class.

Inner class	Nested class
Non-static nested classes are called *inner classes*.	Nested classes that are declared static are called *static nested classes*.
A nested class is a member of its enclosing class	Non-static nested classes (inner classes) have access to other members of the enclosing class

18. List out any two key security features provided by Java programming language.

Object-oriented:

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporates both data and behavior.

Platform Independent:

Java is platform independent. The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on the top of other hardware-based platforms. It has two components:

1. Runtime Environment
2. API(Application Programming Interface)

Java code can be run on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc.

19. What are the benefits of generics in java?

Type-safety: We can hold only a single type of objects in generics. It doesn't allow to store other objects.
Type casting is not required: There is no need to typecast the object.
Compile-Time Checking: It is checked at compile time so problem will not occur at runtime.

20. What are collections in Java?

The **Collection** is a framework that provides an architecture to store and manipulate the group of objects. Collections can achieve all the operations that u perform on a data such as searching, sorting, insertion, manipulation, and deletion. Collection means a single unit of objects. Collection framework provides many interfaces (Set, List, Queue, Deque) and classes.

21. Define anonymous inner class.

Anonymous inner class is a type of inner class which has no name and it can be instantiated only once. It does not have a constructor because it does not have a class name and cannot be static Anonymous inner class always extend a class or implement an interface.

22. Explain: TCP/IP and HTTP.

TCP stands for **Transmission Control Protocol**. It is a transport layer protocol that facilitates the transmission of packets from source to destination. It is a connection-oriented protocol that means it establishes the connection prior to the communication that occurs between the computing devices in a network. This protocol is used with an IP protocol, so together, they are referred to as a TCP/IP.

HTTP:

HTTP stands for Hypertext Transfer Protocol. Hypertext Transfer Protocol is a set of rule which is used for transferring the files like, audio, video, graphic image, text and other multimedia files on the WWW (World Wide Web). The Hypertext Transfer Protocol (HTTP) is application-level protocol for collaborative, distributed, hypermedia information systems. It is the data communication protocol used to establish communication between client and server.

23. What is called URL Connection?

URL stands for Uniform Resource Locator. It takes the form of a string that describes how to find a resource on the Internet. The protocol needed to access the resource and the location of the resource. This class represents an URL and it points to or determines a resource on the World Wide Web.

PART-B
11-Marks

24. Explain the various classes used for network programming.

Networking is a concept of connecting two or more computing devices together so that we can share resources. *Network programming* refers to writing programs that execute across multiple devices (computers), in which the devices are connected to each other via a network. Java encapsulates classes and interfaces to allow low-level communication details

Java Networking is a notion of connecting two or more computing devices together to share the resources. Java program communicates over the network at the **application layer.** java.net package is useful for all the Java networking classes and interfaces.

The java.net package provides support for two protocols. They are as follows:

- **TCP** – Transmission Control Protocol allows reliable communication between two applications. TCP is typically used over the Internet Protocol, which is referred to as TCP/IP.
- **UDP** – User Datagram Protocol is a connection-less protocol that allows packets of data to be transmitted between applications.

Note: Networking in Java is mainly used for sharing the resources and also for centralized software management.

Networking Terminologies

The widely used Java networking terminologies used are as follows:

1. IP Address
2. Protocol
3. Port Number
4. MAC Address
5. Connection-oriented and connection-less protocol
6. Socket

Now let's get into the details of each of these methods.

1. IP Address

The IP address is a unique number assigned to a node of a network *e.g. 192.168.0.1*. It is composed of octets that range from 0 to 255.

2. Protocol

A protocol is a set of rules followed for communication. For example:

- TCP
- FTP
- Telnet
- SMTP
- POP etc.

3. Port Number

The port number uniquely identifies different applications. It acts as a communication endpoint between applications. To communicate between two applications, the port number is used along with an IP Address.

4. MAC Address

A **MAC address** is basically a hardware identification number which uniquely identifies each device on a network. For example, an Ethernet card may have a **MAC address** of 00:0d: 83:b1:c0:8e.

5. Connection-oriented and connection-less protocol

In the connection-oriented protocol, acknowledgment is sent by the receiver. So it is reliable but slow. The example of a connection-oriented protocol is TCP. But, in the connection-less protocol, acknowledgment is not sent by the receiver. So it is not reliable but fast. The example of a connection-less protocol is UDP.

6. Socket

A **socket** in **Java** is one endpoint of a two-way communication link between two programs running on the network. A **socket** is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to.

Inet Address

Inet Address is used to encapsulate both the numerical IP address and the domain name for that address. It can handle both IPv4 and Ipv6 addresses. Below figure depicts the subclasses of Inet Address class.

Socket and Socket Server Class

A socket is used to establish a connection through the use of the port, which is a numbered socket on a particular machine. Socket basically provides a communication mechanism between two computers using Transmission Control Protocol. There are two types of sockets as follows:

- **ServerSocket** is for servers
- The **socket** class is for the client

If you wish to gain more insights on Socket Programming,

URL Class

URL class mainly deals with URL(Uniform Resource Locator) which is used to identify the resources on the internet.

For Example: **https://www.edureka.co/blog**

Here, https: -> Protocol
www.edureka.co -> hostname
/blog - > filename

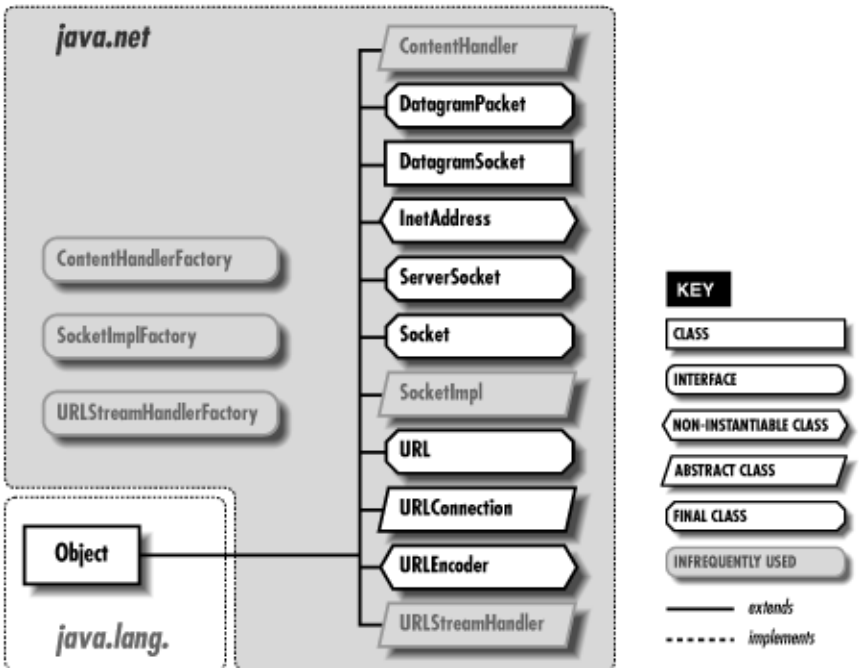
URL Class comprises of various methods to return the URL information of a particular website. Let's now understand various methods of Java URL Class.

1. **getProtocol() :** Returns protocol of URL
2. **getHost() :** Returns hostname(domain name) of the specified URL
3. **getPort() :** Returns port number of the URL specified
4. **getFile() :** Returns filename of the URL

Advantage of Java Networking

1. sharing resources
2. centralize software management

various classes used for network



25. Explain the various components of JDBC in java. Write the steps for using JDBC to access database.

- JDBC is an API(Application programming interface) which is used in java programming to interact with databases.

- *The* classes and interfaces *of JDBC allows application to send request made by users to the specified database*
- JDBC is a Java API to connect and execute the query with the database. It is a part of JavaSE (Java Standard Edition).
- The term JDBC stands for Java Database Connectivity. JDBC is a specification from Sun microsystems.

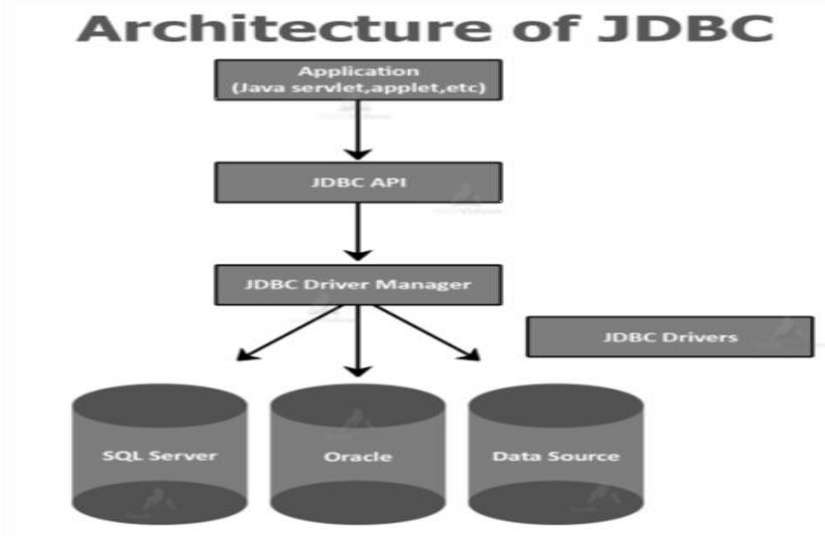
- Using JDBC, we can write programs required to access databases. JDBC and the database driver are capable of accessing databases and spreadsheets. JDBC API is also helpful in accessing the enterprise data stored in a relational database(RDB).

Purpose of JDBC

There are some enterprise applications created using the JAVA EE(Enterprise Edition) technology. These applications need to interact with databases to store application-specific information. Interacting with the database requires efficient database connectivity, which we can achieve using ODBC(Open database connectivity) driver. We can use this ODBC Driver with the JDBC to interact or communicate with various kinds of databases, like Oracle, MS Access, Mysql, and SQL, etc.

Components of JDBC

1.JDBC API: The JDBC API provides various classes, methods, and interfaces that are helpful in easy communication with the database. It also provides two packages that contain the Java SE(Standard Edition) and Java EE(Enterprise Edition) platforms to exhibit the WORA(write once run everywhere) capabilities. There is also a standard in JDBC API to connect a database to a client application.
2. JDBC Driver Manager: The Driver Manager of JDBC loads database-specific drivers in an application. This driver manager establishes a connection with a database. It also makes a database-specific call to the database so that it can process the user request.
3. JDBC Test suite: The Test Suite of JDBC helps to test the operation such as insertion, deletion, updation that the JDBC Drivers perform.
4. JDBC-ODBC Bridge Drivers: The JDBC-ODBC Bridge Driver connects the database drivers to the database. This bridge driver translates the JDBC method call to the ODBC method call. It uses a package in which there is a native library to access ODBC characteristics.



Description of the Architecture

1. Application: Application in JDBC is a Java applet or a Servlet that communicates with a data source.

2. JDBC API: JDBC API provides classes, methods, and interfaces that allow Java programs to execute SQL statements and retrieve results from the database. Some important classes and interfaces defined in JDBC API are as follows:

- DriverManager
- Driver
- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet
- SQL data

3. Driver Manager: The Driver Manager plays an important role in the JDBC architecture. The Driver manager uses some database-specific drivers that effectively connect enterprise applications to databases.

4. JDBC drivers: JDBC drivers help us to communicate with a data source through JDBC. We need a JDBC driver that can intelligently interact with the respective data source.

Steps to connect Java program and database:

1. Loading the Driver

We first need to load the driver or register it before using it in the program. There should be registration once in your program. We can register a driver in any of the two ways:

a. Class.forName(): In this, we load the driver's class file into memory during runtime. There is no need to use a new operator for the creation of an object. The following shows the use of Class.forName() to load the Oracle driver:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

b. DriverManager.registerDriver(): DriverManager is an inbuilt class of Java that comes with a static member register. We call the drivers class' constructor at compile-time. The

following example shows the use of DriverManager.registerDriver() to register the Oracle driver:

```
DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver())
```

2.Create the connections

After loading the driver, we need to establish connections using the following code:

```
Connection con = DriverManager.getConnection(url, user, password)
```

- user: username from which sql command prompt can be accessed.
- password: password from which sql command prompt can be accessed.
- con: reference to Connection interface.
- url : Uniform Resource Locator. We can create it as follows:
- String url = " jdbc:oracle:thin:@localhost:1521:xe"

Where oracle is the database, thin is the driver, @localhost is the IP Address where the database is stored, 1521 is the port number, and xe is the service provider. All three parameters are of String type and the programmer should declare them before calling the function.

3.Create a statement

Once you establish a connection, you can interact with the database. The JDBCStatement, CallableStatement, and PreparedStatement interfaces define the methods that allow us to send the SQL commands and receive data from the database.

Use of JDBC Statement is as follows:

```
Statement statement = con.createStatement()
```

Here, con is a reference to the Connection interface that we used in the previous step.

4.Execute the query

The most crucial part is executing the query. Here, Query is an SQL Query. We can have multiple types of queries. Some of them are as follows:

- The query for updating or inserting tables in a database.
- The query for retrieving data from the database.

The executeQuery() method of the Statement interface executes queries of retrieving values from the database. The executeQuery() method returns the object of ResultSet that we can use to get all the records of a table.

5. Close the connections

Till now, we have sent the data to the specified location. Now, we are about to complete our task. We need to close the connection. By closing the connection, objects of Statement and ResultSet interface are automatically closed. The close() method of Connection interface closes the connection.

Example :

```
con.close();
```

26. Briefly describe about java security.

Java is the most popular **object-oriented programming language**. It provides a variety of salient features that are preferred by the developers. It is the reason that a billion of devices runs on Java.

Java is secure due to the following reasons:

- Java programs run inside a virtual machine which is known as a sandbox.
- Java does not support explicit pointer.
- Byte-code verifier checks the code fragments for illegal code that can violate access right to object.
- It provides java.security package implements explicit security.
- It provides library level safety.
- Run-time security check takes place when we load new code.

Java provides some other features that make Java more secure.

- JVM
- Security API's
- Security Manager
- Auto Memory Management
- No Concept of Pointers
- Compile-time Checking
- Cryptographic Security
- Java Sandbox
- Exception Handling
- ClassLoader

JVM

JVM plays a vital role to provide security. It verifies the byte-code. The JVM provides guarantees that there is no unsafe operation going to execute. It also helps to diminish the possibilities of the programmers who suffer from memory safety flaws.

Security API's

Java class libraries provide several API that leads to security. These APIs contain cryptographic algorithms and authentication protocols that lead to secure communication.

Byte Code

Every time when a user compiles the Java program, the Java compiler creates a class file with Bytecode, which are tested by the JVM at the time of program execution for viruses and other malicious files.

Security Manager

The security manager is responsible for checking the permissions and properties of the classes. It monitors the system resources accessed by the authorized classes. It also controls socket connections.

No Concept of Pointers

Java does not provide support for pointers concept. It is the main security features of Java. The use of pointers may lead to unauthorized read or write operations. Therefore, the user cannot point to any memory locations.

Memory management

Java automatically manages memory which is known as garbage collection. The JVM manages memory itself. The programmers are free from memory management. Hence, there is no chance to fault in memory management.

Compile-time checking

Compile-time checking also makes the Java secure. Consider a scenario in which an unauthorized method is trying to access the private variable, in this case, the JVM gives the compile-time error. It prevents the system from the crash.

Cryptographic Security

Java provides a class named **java.secrurity.SourceCode** that also provides security. If we get code from other sources, we should check from where the code is coming. The class maintains the source information and provides guarantees to keep a digital signature and cryptographic security.

Java Sandbox

Java Sandbox is a major component of security consideration. It is a restricted area where applets are run. Java does not provide system resources without check if an applet is to be run.

Exception Handling

The exception handling feature adds more security in Java. The feature reports the error to the programmer during the runtime. The code will not run until the programmer will not rectify it.

Java ClassLoader

There are a number of class loaders present in JVM. It provides and maintains namespaces for specific classes. The advantage of the Class Loader is that the untrusted classes would not behave like a trusted one.

27. Explain the concept of object and inner classes with example

Java inner class or nested class is a class which is declared inside the class or interface.Inner class means one class which is a member of another class.

We use inner classes to logically group classes and interfaces in one place so that it can be more readable and maintainable. Additionally, it can access all the members of outer class including private data members and methods.

Java inner class is defined inside the body of another class. Java inner class can be declared private, public, protected, or with default access whereas an outer class can have only public or default access.

Syntax of Inner class

```
class Java_Outer_class{
  //code
class Java_Inner_class{
  //code
 }
}
```

Advantage of java inner classes

There are basically three advantages of inner classes in java. They are as follows:

1) Nested classes represent a special type of relationship that is **it can access all the members (data members and methods) of outer class** including private.

2) Nested classes are used **to develop more readable and maintainable code** because it logically group classes and interfaces in one place only.

3) **Code Optimization**: It requires less code to write.

Types of Nested classes

There are two types of nested classes non-static and static nested classes.The non-static nested classes are also known as inner classes.

- Non-static nested class (inner class)
    1. Method inner class
    2. Anonymous inner class
    3. Local inner class
- Static nested class

Non-static nested class

A Non-static nested class is a class within another class. It has access to members of the enclosing class (outer class). It is commonly known as inner class. Since the inner class exists within the outer class.We must instantiate the outer class first, in order to instantiate the inner class.

Method Local inner classes

Inner class can be declared within a method of an outer class. In the following example, Inner is an inner class in outerMethod().Method Local inner classes can't use local variable of outer method until that local variable is not declared as final.

```
class Outer {

    void outerMethod() {

        System.out.println("inside outerMethod");

        // Inner class is local to outerMethod()

        class Inner {

            void innerMethod() {

                System.out.println("inside innerMethod");
```

```
            }
        }
        Inner y = new Inner();
        y.innerMethod();
    }
}
class MethodDemo {
    public static void main(String[] args) {
        Outer x = new Outer();
        x.outerMethod();
    }
}
```

Output

Inside outerMethod

Inside innerMethod

local inner classes

Local inner classes a class is defined in a method body is known as local inner class.Since the local inner class is not associated with Object, we can't use private, public or protected access modifiers with it. The only allowed modifiers are abstract or final.

A local inner class can access all the members of the enclosing class and local final variables in the scope it's defined. Additionally, it can also access a non-final local variable of the method in which it is defined, but it cannot modify them.So if you try to print non-final local

variable's value it will be allowed but if you try to change its value from inside method local inner class, you will get compile time Error.

Anonymous inner class

- A local inner class without name is known as anonymous inner class. An anonymous class is defined and instantiated in a single statement.

- Anonymous inner class always extend a class or implement an interface. Since an anonymous class has no name, it is not possible to define a constructor for an anonymous class.

- Anonymous inner classes are accessible only at the point where it is defined. It's a bit hard to define how to create an anonymous inner class.

Benefits of Java Inner Class

- If a class is useful to only one class, it makes sense to keep it nested and together. It helps in the packaging of the classes.
- Java inner classes implements encapsulation. Note that inner classes can access outer class private members and at the same time we can hide inner class from outer world.
- Keeping the small class within top-level classes places the code closer to where it is used and makes the code more readable and maintainable.

Static nested class

The nested class is static, then it's called a static nested class. Static nested classes can access only static members of the outer class. A static nested class is the same as any other top-level class and is nested for only packaging convenience.

A static class object can be created with the following statement.

```
OuterClass.StaticNestedClass nestedObject =
    new OuterClass.StaticNestedClass();
```

Object cloning refers to the creation of an exact copy of an object. It creates a new instance of the class of the current object and initializes all its fields with exactly the contents of the corresponding fields of this object.

Syntax

```
protected Object clone() throws CloneNotSupportedException
```

The **object cloning** is a way to create exact copy of an object. The clone() method of Object class is used to clone an object.

The **java.lang.Cloneable interface** must be implemented by the class whose object clone we want to create. If we don't implement Cloneable interface, clone() method generates **CloneNotSupportedException**.

The **clone() method** saves the extra processing task for creating the exact copy of an object. If we perform it by using the new keyword, it will take a lot of processing time to be performed that is why we use object cloning.

Advantage of Object cloning

Although Object.clone() has some design issues but it is still a popular and easy way of copying objects. Following is a list of advantages of using clone() method:

- You don't need to write lengthy and repetitive codes. Just use an abstract class with a 4- or 5-line long clone() method.
- It is the easiest and most efficient way for copying objects, especially if we are applying it to an already developed or an old project. Just define a parent class, implement Cloneable in it, provide the definition of the clone() method and the task will be done.
- Clone() is the fastest way to copy array.

Types of Cloning in Java

Cloning in Java can be grouped into two categories:

1. Shallow Cloning
2. Deep Cloning

Shallow Cloning

The default implementation of Java Object clone() method is using shallow copy. It's using reflection API to create the copy of the instance. The below code snippet showcase the shallow cloning implementation.

Deep cloning

In deep cloning, we have to copy fields one by one. If we have a field with nested objects such as List, Map, etc. then we have to write the code to copy them too one by one. That's why it's called deep cloning or deep copy.

```
/ A Java program to demonstrate

// shallow copy using clone()

import java.util.ArrayList;

 // An object reference of this class is

// contained by Test2

class Test {

    int x, y;

}

 // Contains a reference of Test and

// implements clone with shallow copy.

class Test2 implements Cloneable {

    int a;

    int b;
```

```
    Test c = new Test();
    public Object clone() throws CloneNotSupportedException
    {
        return super.clone();
    }
}

// Driver class
public class Main {
    public static void main(String args[])
        throws CloneNotSupportedException
    {
        Test2 t1 = new Test2();
        t1.a = 10;
        t1.b = 20;
        t1.c.x = 30;
        t1.c.y = 40;

        Test2 t2 = (Test2)t1.clone();
         // Creating a copy of object t1
```

```
        // and passing it to t2
        t2.a = 100;
         // Change in primitive type of t2 will
        // not be reflected in t1 field
        t2.c.x = 300;
         // Change in object type field will be
        // reflected in both t2 and t1(shallow copy)
        System.out.println(t1.a + " " + t1.b + " " + t1.c.x
                           + " " + t1.c.y);
        System.out.println(t2.a + " " + t2.b + " " + t2.c.x
                           + " " + t2.c.y);
    }
}
```

Output

```
10 20 300 40
100 20 300 40
```

28. Discuss with an example the advantages of using Input Output Packages.

A java package is a group of similar types of classes, interfaces and sub-packages.Package in java can be categorized in two form, built-in package and user-defined package.There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Advantages of using Java Packages

- Make easy searching or locating of classes and interfaces.
- Avoid naming conflicts. For example, there can be two classes with the name Student in two packages, university.csdept.Student and college.itdept.Student
- Implement data encapsulation (or data-hiding).
- Provide controlled access: The access specifiers protected and defaults have access control on package level. A member declared as protected is accessible by classes within the same package and its subclasses. A member without any access specifier that is default specifier is accessible only by classes in the same package.
- Reuse the classes contained in the packages of other programs.
- Uniquely compare the classes in other packages.

Java.io Package in Java

This package provides for system input and output through data streams, serialization and the file system.

Java I/O (Input and Output) is used *to process the input* and *produce the output*.

Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.

Stream

A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In Java, 3 streams are created for us automatically. All these streams are attached with the console.

1) System.out: standard output stream

2) System.in: standard input stream

3) System.err: standard error stream

Let's see the code to print **output and an error** message to the console.

```
1. System.out.println("simple message");
2. System.err.println("error message");
```

Let's see the code to get **input** from console.

```
1. int i=System.in.read();//returns ASCII code of 1st character
2. System.out.println((char)i);//will print the character
```

29. Explain the utility packages.

Java.util package contains the collection's framework, legacy collection classes, event model, date and time facilities, internationalization, and miscellaneous utility classes (a string tokenizer, a random-number generator, and a bit array). Here is the list of most commonly used top ten Java utility classes:

1. Java Arrays Class

Java.util package provides an **Arrays class** that contains a static factory that allows arrays to be viewed as lists. The class contains various methods for manipulating arrays like sorting and searching. The methods in this class throw a NullPointerException, if the specified array reference is null.

2. Java Vector Class

Java.util package provides a **vector class** that models and implements vector data structure. A vector is a sequence container implementing array that can change in size. Unlike an array, the size of a vector changes automatically when elements are appended or deleted. It is similar to ArrayList, but with two differences:

- Vector is synchronized.
- Vector contains many legacy methods that are not part of the collections framework.

3. Java LinkedList Class

Java.util package provides a **LinkedList class** that has Doubly-linked list implementation of the List and Deque interfaces. The class has all optional list operations, and permits all elements (including null). Operations that index into the list will traverse the list from the beginning or the end, whichever is closer to the specified index.

4. Java Calendar Class

Java.util package provides a **Calendar class** that represents a specific instant in time, with millisecond precision.

5. Java Collections Class

Java.util package provides a **Collections class** which consists exclusively of static methods that operate on or return collections. It contains polymorphic algorithms that operate on collections, "wrappers", which return a new collection backed by a specified collection. The methods of this class all throw a NullPointerException if the collections or class objects provided to them are null.

6. Java HashMap Class

A Hashtable models and implements the Map interface. This implementation provides all the optional map operations and permits null values and the null key. A **HashMap class** is roughly equivalent to Hashtable, except that it is unsynchronized and permits nulls. This class makes no guarantees as to the order of the map. It does not guarantee that the order will remain constant over time.

7. Java Random Class

Java.util package provides a **Random class**. An instance of this class is used to generate a stream of pseudorandom numbers. The class uses a 48-bit seed, which is modified using a linear congruential formula. The algorithms implemented by class Random use a protected utility method that on each invocation can supply up to 32 pseudorandomly generated bits.

8. Java UUID Class

Java.util package provides a **UUID class** that represents an immutable universally unique identifier (UUID). A UUID represents a 128-bit value. It is used for creating random file names, session id in web application, transaction id etc. There are four different basic types of UUIDs: time-based, DCE security, name-based, and randomly generated UUIDs.

9. Java Scanner Class

Java.util package provides a **Scanner class**. A simple text scanner parse primitive types and strings using regular expressions. A Scanner breaks its input into tokens using a delimiter pattern, which by default matches whitespace. The resulting tokens may then be converted into values of different types using the various next methods.

A scanning operation may block waiting for input and not safe for multi-threaded use without external synchronization.

10. Java ArrayList Class

Java.util package provides an **ArrayList class** that provides resizable-array implementation of the list interface. It implements all optional list operations and permits all elements including null. Along with this, the class provides methods to manipulate the size of the array that is

used internally to store the list. This class is almost equivalent to Vector class except the implementation is not synchronized.

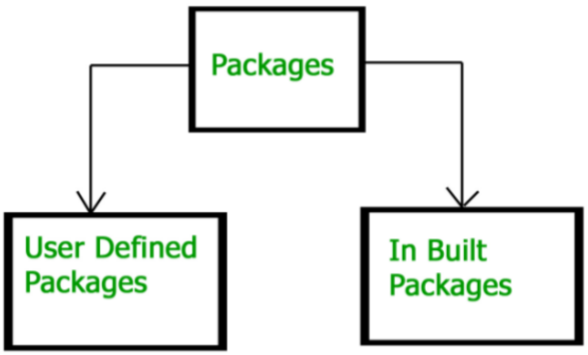
30. Explain packages in Java and list out all packages with short description.

Package is a mechanism to encapsulate a group of classes, sub packages and interfaces. A package is a namespace that organizes a set of related classes and interfaces. Packages are used for:

- Preventing naming conflicts. For example there can be two classes with name Employee in two packages, college.staff.cse.Employee and college.staff.ee.Employee
- Making searching/locating and usage of classes, interfaces, enumerations and annotations easier
- Providing controlled access: protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only.
- Packages can be considered as data encapsulation (or data-hiding).

All we need to do is put related classes into packages. After that, we can simply write an import class from existing packages and use it in our program. A package is a container of a group of related classes where some of the classes are accessible are exposed and others are kept for internal purpose.
We can reuse existing classes from the packages as many time as we need it in our program.

Types of packages:



Built-in Packages

These packages consist of a large number of classes which are a part of Java **API**.Some of the commonly used built-in packages are:

1) **java.lang:** Contains language support classes(e.g classed which defines primitive data types, math operations). This package is automatically imported.
2) **java.io:** Contains classed for supporting input / output operations.
3) **java.util:** Contains utility classes which implement data structures like Linked List, Dictionary and support ; for Date / Time operations.
4) **java.applet:** Contains classes for creating Applets.

5) **java.awt:** Contain classes for implementing the components for graphical user interfaces (like button , ;menus etc).
6) **java.net:** Contain classes for supporting networking operations.

User-defined packages

These are the packages that are defined by the user. First we create a directory **myPackage** (name should be same as the name of the package).

Illustration of user-defined packages:

Creating our first package:
File name – ClassOne.java

```
package package_name;



public class ClassOne {

    public void methodClassOne() {

        System.out.println("Hello there its ClassOne");

    }

}
```

Creating our second package:
File name – ClassTwo.java

```
package package_one;



public class ClassTwo {

    public void methodClassTwo(){

        System.out.println("Hello there i am ClassTwo");

    }
```

```
}
```

Making use of both the created packages:
File name – Testing.java

```
import package_one.ClassTwo;

import package_name.ClassOne;

  public class Testing {

    public static void main(String[] args){

        ClassTwo a = new ClassTwo();

        ClassOne b = new ClassOne();

        a.methodClassTwo();

        b.methodClassOne();

    }

}
```

Output:

Hello there i am ClassTwo

Hello there its ClassOne

Advantages of using a package

- **Re-usability:** The classes contained in the packages of another program can be easily reused
- **Name Conflicts:** Packages help us to uniquely identify a class, for example, we can have *company.sales.Employee* and *company.marketing.Employee* classes
- **Controlled Access:** Offers access protection such as protected classes, default classes and private class

- **Data Encapsulation:** They provide a way to hide classes, preventing other programs from accessing classes that are meant for internal use only
- **Maintainance:** With packages, you can organize your project better and easily locate related classes

Difference between Method Overloading and method Overriding.

S.No	Method Overloading	Method Overriding
1.	Method overloading is a compile time polymorphism.	Method overriding is a run time polymorphism.
2.	It is occur within the class	While it is performed in two classes with inheritance relationship.
3.	Method overloading may or may not require inheritance.	While method overriding always needs inheritance.
4.	In this, methods must have same name and different signature.	While in this, methods must have same name and same signature.
5.	In case of method overloading, *parameter must be different*.	In case of method overriding, *parameter must be same*.

31. Briefly Explain the generics with example in java.

Generics means **parameterized types. Java Generics** is a set of related methods or a set of similar types. Generics allow types Integer, String, or even user-defined types to be passed as a parameter to classes, methods, or interfaces. Generics are mostly used by classes like HashSet or HashMap.

Object is the superclass of all other classes and Object reference can refer to any type object.

Generics is a term that denotes a set of language features related to the definition and use of Generic types and methods. Java Generic methods differ from regular data types and methods

Types of Java Generics

There are 4 different ways Generics can be applied to in Java and they are as follows:

1. Generic Type Class
2. Generic Interface
3. Generic Method
4. Generic Constructor

Generic Type Class

A class is said to be Generic if it declares one or more type variables. These variable types are known as the type parameters of the Java Class.

Generic Interface

An interface in Java refers to the abstract data types. They allow Java collections to be manipulated independently from the details of their representation.

```
class Main {
  public static void main(String[] args) {

    // initialize the class with Integer data
    DemoClass demo = new DemoClass();

    // generics method working with String
    demo.<String>genericsMethod("Java Programming");

    // generics method working with integer
    demo.<Integer>genericsMethod(25);
  }
}
```

```
class DemoClass {  // creae a generics method  public <T> void
genericsMethod(T data) {    System.out.println("Generics Method:");
System.out.println("Data Passed: " + data);  }}
```

Output

```
Generics Method:
Data Passed: Java Programming
Generics Method:
Data Passed: 25
```

Generic Methods

Generic methods are much similar to generic classes. They differ from each other in only one aspect that the *scope or type information is inside the method only.* Generic methods introduce their own type parameters.

Generic Constructor

A constructor is a block of code that initializes the newly created object. A **constructor** resembles an instance method in J**ava** but it's not a method as it doesn't have a return type. **The constructor** has the same name as the class and looks like this in J**ava** code.

Advantages of Generics in Java

1. Code Reusability

You can compose a strategy or a class or an interface once and use for any type or any way that you need.

2. Individual Types Casting isn't required

Basically, you recovered information from ArrayList every time, you need to typecast it. Typecasting at each recovery task is a major migraine. To eradicate that approach, generics were introduced.

3. Implementing a non-generic algorithm

It can calculate the algorithms that work on various sorts of items that are type safe as well.